LECTURE 30

CLASSES P, NP & NP-COMPLETE

1. Polynomial running time: Defined as running time $O(n^k)$, for some integer constant k.

All the algorithms studied so far have polynomial running time, as defined.

The class of all problems which can be solved in polynomial running time is designated as **P**. These problems are considered to be **tractable**.

Therefore now we can say that all the computing problems which we have studied so far (sorting, DFS, shortest path ... *et cetera*) are in **P**.

[Note: It is implicit that we will always use an efficient algorithm to solve a problem. Therefore we can refer to problems rather than algorithms.]

2. Non-determinism: When a transition function (at the heart of the automaton) is of the following kind:

$\delta(\ \mathbf{q_i}\ ,\ a\ \ldots\ ) = \{\ \mathbf{q_j}\ ,\ \mathbf{q_k}\ ,\ \ldots.\ \}$ **Note set of next states on RHS!**

Question: In such an automaton, what is the actual "next state"?

Answer: Any one of $\{\ \mathbf{q_j}\ ,\ \mathbf{q_k}\ ,\ \ldots.\ \}$ ... "whichever one works best!" This is the essence of **non-determinism** in the context of computer algorithms.

In practice, that leaves us with $\mathbf{k^n}$ **different possible combinations** to find the one which works, for some constant integer k. This number arises when we enumerate all possible sequences of states which can arise.

So polynomial running time on a non-deterministic automaton is NOT actually polynomial running time as defined above (to introduce class **P**).

The class of problems solvable in polynomial time on a non-deterministic automaton is called **NP**, (N for **non-deterministic**).

By definition, we can say that: $P \subseteq NP$.

3. Another definition of NP [CLRS]:

We are given a certificate, say **V** – which is like a trial solution. We only need to plug **V** into the problem and verify that **V** solves the problem.

Essentially, V gives us one possible trajectory out of $\mathbf{k^n}$, whose correctness can be **verified** in polynomial time. In general, verifying that ABC... is a correct solution is tractable even if finding a correct solution is not.

The two definitions of class NP are equivalent.

Big unanswered question: P = NP?

4. Polynomial time reduction & reducibility:

Suppose problem P_1 is solvable in polynomial time $O(p_1(n))$.

Suppose now that any occurrence of another problem P_2 can be mapped – or **reduced** – to an occurrence of P_1 in polynomial time $O(p_2(n))$.

Then P_2 can be solved in time $O(p_1(p_2(n)))$.

Justification: In time $O(p_2(n))$, the output of the reduction process – that is, the reduced problem – has length $O(p_2(n))$. That is the input to P_1.

Example: If $p_1(n) = n^3$ and $p_2(n) = n^4$, then $p_1(p_2(n)) = (n^4)^3 = n^{12}$.

∵ Any polynomial of any polynomial must necessarily be a polynomial. In simple language, polynomial time reducibility of P_2 to P_1 implies that:

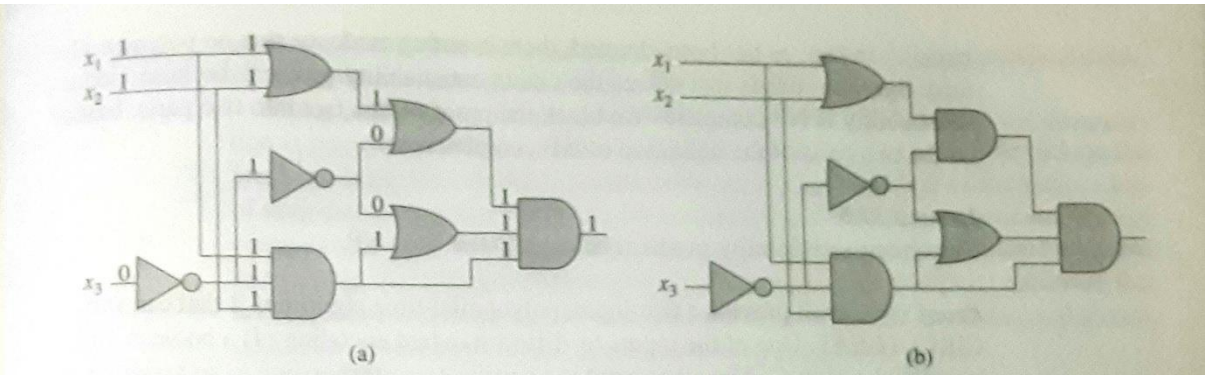
If P_1 is in **P**, then P_2 is also in **P**.

In notation, this is represented as: $P_2 \leq_P P_1$.

5. Boolean circuit satisfiability problem [CLRS]:

Given a Boolean combinational circuit C composed of AND, OR and NOT gates, is the circuit **satisfiable**? In other words, is there an assignment of Boolean values to the inputs of C which will make the circuit output = 1?

Example:



The circuit on the left is satisfiable (as shown), but the one on the right is not.

How can one prove such statements? In general terms, by exhaustive enumeration and testing of all possible inputs. Exponential time!

IF Boolean circuit satisfiability (BCS) is in class **P**, then so is any problem Q which is polynomially reducible to BCS, that is $Q \leq_P$ BCS. Why? Because the reduction of Q and BCS are then both in **P** → polynomial of a polynomial.

6. NP-COMPLETE problems

A problem Q is **NP-COMPLETE** if:

(1) $Q \in NP$, and

(2) any other problem $Q' \in NP$ is polynomially reducible to Q, that is: $Q' \leq_P Q$.

→ BCS is an **NP-COMPLETE** problem.

[Recall that digital computers can be viewed as giant Boolean circuits.]

[**GK**]

(1) "Languages" ≡ computational problems.

(2) Major result proved at IITK a few years ago: **PRIMES IS IN P**.